

MODULE ALGORITHMIQUE ET STRUCTURES DE DONNÉES AVANCÉES

COMPLEXITÉ ALGORITHMIQUE

©B. DERDOURI

CM- Complexité Algorithmique Algo3 L3-Info Semestre5 DepInfo-Univ-CERGY

06/09/2023

1

PLAN

- NOTION de Complexités des algorithmes
 - Position du problème & Objectif
 - Comment mesurer l'efficacité d'un algorithme ?
 - Choix d'un système de mesure & Paramètres de la fonction coût
 - Règles de calcul
 - Différentes notions de complexité d'un algorithme
- Exemples
- Rappel : Approche Diviser Pour Mieux Régner
 - Complexité d'un algorithme récursif
 - Exemple calcul de $n!$
- Principales classes de complexité

©B. DERDOURI

CM- Complexité Algorithmique Algo3 L3-Info Semestre5 DepInfo-Univ-CERGY

06/09/2023

2

NOTION DE COMPLEXITÉS DES ALGORITHMES

- **Rappel**
- Un algorithme est la composition d'un nombre fini d'actions (instructions).
- Une action étant formée d'un nombre fini d'opérations.
- chaque opération étant :
 - Définie de façon rigoureuse et non ambiguë
 - Effectivement réalisable sur une machine.
- **Définition**
- L'algorithmique est la science qui permet de trouver un bon algorithme pour un problème donné, d'en faire **la preuve et d'en analyser la complexité**.

POSITION DU PROBLÈME

- L'exécution d'un algorithme nécessite l'utilisation des ressources de l'ordinateur :
- **Temps de calcul** pour exécuter les opérations élémentaires :
 - Lecture des données,
 - Écriture des résultats,
 - Affectations,
 - Opérations arithmétiques,
 - Opérations de comparaisons,
 - Etc ...
- **Le temps d'exécution** dépend du nombre de ces opérations → **la complexité en temps (temporelle)**
- Et **occupation de la mémoire** pour contenir et manipuler l'algorithme et ses données.
- **L'espace mémoire** dépend de la taille des données → **la complexité en mémoire (spatiale)**.

Par taille des données, on entend le nombre d'octets nécessaires pour les coder, et en général on admet qu'il est **proportionnel au nombre des données**.

OBJECTIF DU COURS

Proposer des méthodes qui, pour la résolution d'un problème donné, permettent d'estimer le coût d'un algorithme, de comparer deux algorithmes différents réalisant le même traitement (sans avoir à les programmer effectivement).

EXEMPLES

- 1) Si on lance le calcul de la factorielle de $n=100$, combien de temps faudra-t-il pour attendre le résultat ?
- 2) Quel algorithme de tri vaut-t-il mieux utiliser pour trier un tableau.
- 3) Quel algorithme de recherche vaut-t-il mieux utiliser pour rechercher un élément donné dans un tableau composé de n éléments ?

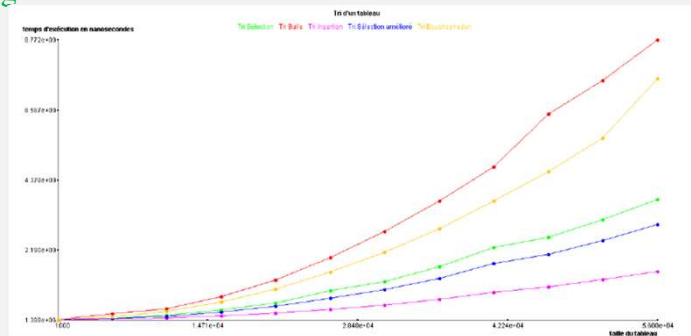
* Nature du tableau trié ou pas .

L'évaluation de la complexité peut se faire à plusieurs niveaux :

- au niveau purement algorithmique, par l'analyse et le calcul.
- au niveau de l'exécution du programme expérimentalement.

COMPLEXITÉ EXPÉRIMENTALE

- Il est possible d'évaluer de façon expérimentale le **temps d'exécution des programmes**.



- Cette évaluation expérimentale dépend beaucoup des langages de programmation, ordinateurs et systèmes d'exploitation utilisés.

COMMENT MESURER L'EFFICACITÉ D'UN ALGORITHME ?

- Le temps d'exécution d'un algorithme dépend :**

- De la machine
- Du langage de programmation
- Du compilateur
- Des données
- Etc

- On aimerait pouvoir dire : sur toute machine et quel que soit le langage de programmation, l'algorithme A1 est meilleur que l'algorithme A2 pour des données de grande taille.

COMMENT MESURER L'EFFICACITÉ D'UN ALGORITHME ?

- **On voudrait que la complexité :**
- ne dépende pas de l'ordinateur
- ne dépende pas du langage de programmation
- ne dépende pas du programmeur
- soit indépendante des détails de l'implémentation
- Etc...

CHOIX D'UN SYSTÈME DE MESURE

1) Temps d'exécution

- Chaque instruction a son propre temps d'exécution.
- **Exemple :**
- L'affectation est quasi-instantanée,
- L'addition et la soustraction sont très rapides,
- La multiplication un peu moins,
- La division est relativement lente,
- etc.
- **On ne peut donc pas considérer chaque instruction de la même façon.**
Donc là aussi, on aura tendance à ne pas compter ce qui est négligeable.

CHOIX D'UN SYSTÈME DE MESURE

• Exemple 1

- Dans un problème qui fait beaucoup de calculs, on négligera les affectations.
- Par contre, dans un algorithme de tri, qui ne fait que des tests et affectations, cela serait ridicule !

CHOIX D'UN SYSTÈME DE MESURE

• Exemple 2: algorithmes d'échange de deux valeurs entières

Solution 1 :

$$\text{tmp} \leftarrow x \qquad x \leftarrow y \qquad y \leftarrow \text{tmp}$$

La première méthode utilise une variable supplémentaire et réalise 3 affectations

Solution 2 :

$$x \leftarrow y-x \qquad y \leftarrow y-x \qquad x \leftarrow y+x$$

La deuxième méthode n'utilise que les deux variables dont on veut échanger les valeurs, mais réalise 3 affectations et 3 opérations

RECHERCHE SÉQUENTIELLE DANS UN TABLEAU QUELCONQUE

• Version 1:

Fonction itRechSeq_v1 (n, X, tab) → booléen

P.F n : entier (E) Résultat de la fonction
 X : TElement (E) Vrai si X ∈ tab[1..n]
 tab : Tableau de TElement (E) Faux si X ∉ tab[1...n]

Début

```

i ← 1
trouve ← faux
tant que (i ≤ n) et (non trouve) faire
    si tab[i] = X alors
        trouve ← vrai // trouve càd X ∈ tab[1..i]
    sinon
        i ← i + 1 // non trouve càd X ∉ tab[1...i - 1]
    fsi
ftq
Renvoyer trouve // (ou i ≤ n)

```

Fin

RECHERCHE SÉQUENTIELLE DANS UN TABLEAU QUELCONQUE

• Version 2 :

• Remarque : non trouve ↔ X ∉ tab[1...i]

Fonction itRechSeq_v2(n, X, tab) → booléen

P.F n : entier (E) Résultat de la fonction
 X : TElement (E) Vrai si X ∈ tab[1..n]
 tab : Tableau de TElement (E) Faux si X ∉ tab[1...n]

Début

```

i ← 1
tant que (i ≤ n) et tab[i] ≠ X faire
    i ← i + 1
ftq
Renvoyer i ≤ n

```

Fin

RECHERCHE SÉQUENTIELLE DANS UN TABLEAU QUELCONQUE

- Version 3 : Recherche avec sentinelle

```

Fonction itRechSeqSent(n, X, tab) → booléen
P.F      n : entier (E)      Résultat de la fonction
           X : TElement (E)    Vrai si X ∈ tab[1..n]
           tab : Tableau de TElement (E)  Faux si X ∉ tab[1...n]

Début
    i ← 1
    tab[n+1] ← X // Ajout d'une sentinelle
    tant que tab[i] ≠ X faire
        i ← i + 1
    ftq
    Renvoyer i ≤ n
Fin
  
```

PARAMÈTRES DE LA FONCTION COÛT

- L'analyse de la complexité consiste à déterminer une fonction associant un coût (en unité de temps ou de mémoire) à chaque entrée soumise à l'algorithme.
- Le plus souvent on associe le coût à un paramètre entier n qui résume la taille des données.

PARAMÈTRES DE LA FONCTION COÛT

Exemple : Le calcul de la factorielle

- **Fonction factorielle(n) → Entier**
- P.F n : Entier (E)
- **début**
 - `f ← 1;`
 - pour i variant de 2 à n pas de 1 faire
 `f ← f*i;`
 - finpour
 - renvoyer f
- **Fin**
- Le paramètre de complexité est la valeur de n

PARAMÈTRES DE LA FONCTION COÛT

Exemples

1. Pour la recherche d'un élément dans une liste : n = taille de la liste,
2. Pour le tri d'un tableau : n = nombre d'éléments,
3. Pour le calcul d'un terme d'une suite définie par une relation de récurrence (par exemple : suite, fibonacci, ...) : n = indice du terme.

RÈGLES DE CALCUL

- On compte le nombre d'opérations caractéristiques de l'algorithme, en utilisant les règles du type suivant :
 - ♦ le temps d'exécution (T.E) d'une affectation ou d'un test est considéré comme constant
 - ♦ le temps d'exécution d'une séquence d'actions est la somme des T.E des actions qui la composent

RÈGLES DE CALCUL

- ♦ $TE(\text{ si Cond alors action1 sinon action2}) \leq TE(\text{Cond}) + \max(TE(\text{action1}), TE(\text{action2}))$

Majoration du coût en estimant celui de la branche la plus coûteuse.

- On peut aussi faire un calcul en moyenne si l'on connaît les probabilités de réalisation de la condition

$$TE(\text{si Cond alors action1 sinon action2}) = TE(\text{Cond}) + TE(\text{action1}) * \text{Prob}(\text{Cond}=\text{Vrai}) + TE(\text{action2}) * \text{Prob}(\text{Cond}=\text{faux})$$

RÈGLES DE CALCUL

- ◆ Le T.E d'une boucle est $\sum_i TE(i)$ où i est la variable de contrôle de la boucle et $TE(i)$ le temps d'exécution de la i ème itération.

(Dénombrer le nombre de passages dans la boucle en tenant compte de la condition d'arrêt éventuelle).

- ◆ Pour les fonctions récursives, il faudra résoudre une équation de récurrence
- ◆ ...

DIFFÉRENTES NOTIONS DE COMPLEXITÉ D'UN ALGORITHME

Soient un algorithme A et $D(n)$ l'ensemble des jeux possibles de données de taille n .

Notons $\text{Coût}(A(d))$ = la complexité en temps de l'algorithme A sur la donnée d appartenant à $D(n)$.

On peut alors considérer :

- La complexité dans le pire des cas :
- $C_Max(n) = \text{Max} \{ \text{coût}(A(d)) / d \in D(n) \}$.
- La complexité dans le meilleur des cas :
- $C_Min(n) = \text{Min} \{ \text{coût}(A(d)) / d \in D(n) \}$.

DIFFÉRENTES NOTIONS DE COMPLEXITÉ D'UN ALGORITHME

- Les complexités dans **le meilleur et dans le pire des cas** donnent des indications sur les bornes extrêmes de la complexité de l'algorithme sur les données de taille n.
- Leur détermination nécessite en général la construction de données particulières, qui forcent l'algorithme à se comporter de façon extrême.
- La complexité dans **le pire des cas**, qui donne une borne supérieure du temps d'exécution, est particulièrement utile car elle permet de donner une estimation de la taille maximale des données qui pourront être traitées par l'algorithme.

DIFFÉRENTES NOTIONS DE COMPLEXITÉ D'UN ALGORITHME

- Les cas extrêmes ne sont pas les plus fréquents et dans la pratique on souhaite savoir le comportement général de l'algorithme.

- D'où l'introduction de la complexité en moyenne des cas :

$$C_Moy(n) = \sum_{d \in D(n)} p(d) * coût(A(d))$$

- Où $p(d)$ est la probabilité que l'on ait la donnée d en entrée de l'algorithme.

DIFFÉRENTES NOTIONS DE COMPLEXITÉ D'UN ALGORITHME

• Remarques

* Si toutes les données sont équiprobables alors la complexité en moyenne s'exprime simplement en fonction du nombre de données de taille n .

- $C_Moy(n) = (1/|D(n)|) * \sum \text{coût}(A(d))$. (en effet, $p(d)=1/|D(n)|$).

* En général, les données n'ont pas toutes la même probabilité et la définition de la complexité en moyenne nécessite l'introduction d'un modèle probabiliste lié au problème.

Il n'est pas toujours facile de déterminer un modèle de probabilités adéquat au problème.

DIFFÉRENTES NOTIONS DE COMPLEXITÉ D'UN ALGORITHME

• Relation entre les complexités :

$$C_Min(n) \leq C_Moy(n) \leq C_Max(n)$$

• Cas particulier

Si le comportement de l'algorithme ne dépend que de la taille des données, alors ces trois quantités sont confondues.

EXEMPLE 1

- //Somme des n premiers nombre.
- **Fonction somme(n) → Entier**
- P.F n : Entier (E)
- Début
- $S \leftarrow 0$
- Pour i variant de 1 à n faire
- $S \leftarrow s + i$
- FinPour
- Renvoyer s
- Fin
- Étudier la complexité de la fonction somme en nombre d'additions.

EXEMPLE1 (SUITE)

- **Remarque :**
- Le calcul de la fonction ne dépend pas de la nature des nombres à sommer, mais elle dépend uniquement de n.
- Donc $C_Min(n) = C_Moy(n) = C_Max(n)$ = de l'ordre de n
- En effet, pour chaque valeur de i on effectue une addition et comme i varie de 1 à n, on effectue donc au total n additions.

EXEMPLE 2

- **//Produit scalaire de deux vecteurs dans $\mathbb{R}^n$.**
- **Fonction produitScalaire(n , $vec1$, $vec2$) $\rightarrow$ Réel**
- **P.F** n : Entier ($\mathbb{E}$)
- $Vec1, vec2$: Vecteur ($\mathbb{E}$)
- Début
- $S \leftarrow 0$
- Pour i variant de 1 à n faire
- $S \leftarrow S + vec1[i] * vec2[i]$
- FinPour
- Renvoyer S
- Fin
- Étudier la complexité de la fonction produitScalaire en nombre de multiplications.

EXEMPLE2 (SUITE)

- **Remarque :**
- La fonction produitScalaire ne dépend pas de la nature des vecteurs, mais elle dépend uniquement de leurs dimensions n .
- Donc $C_Min(n) = C_Moy(n) = C_Max(n) = n$
- En effet, pour chaque valeur de i on effectue une multiplication et comme i varie de 1 à n , on effectue donc au total n multiplications.

EXEMPLE 3

- **//Multiplication des matrices carrées dans $E^n \times E^n$**
- Soient $A=(a_{ij})$ et $B=(b_{ij})$ deux matrices carrées $n \times n$ à coefficients dans un ensemble E .
- La matrice produit $C=A*B$ est définie comme suit : $C_{ij} = \sum (A_{ik}) * (B_{kj})$.
- Procédure produitMatricielle(n, A, B, C)
- **Début**
 - Pour i variant de 1 à n faire
 - Pour j variant de 1 à n faire
 - $s \leftarrow 0$
 - Pour k variant de 1 à n faire
 - $s \leftarrow s + A[i, k] * B[k, j]$
 - Finpour
 - $C[i, j] \leftarrow s$
 - Finpour
 - **Fin**
 - Étudier la complexité de la procédure produitMatricielle en nombre de multiplications.

EXEMPLE3 (SUITE)

- **Remarque :**
- La procédure produitMatricielle ne dépend pas de la nature des coefficients des matrices, mais elle dépend uniquement de leurs dimensions n .
- Donc $C_Min(n) = C_Moy(n) = C_Max(n) = \sum \sum \sum 1 = n^3$
- En effet, pour chaque valeur de k , on effectue une multiplication.

EXEMPLE 4

- En ne comptabilisant que les comparaisons, étudier la complexité de la fonction qui recherche de manière séquentielle un élément dans un tableau non triés composés de n éléments.

```

Fonction rechercheSeq(n,elt,tab) → boolean
Début
    i ← 1
    Tantque (i≤n) et (elt <> tab[i]) faire
        i ← i + 1
    Fintq
    renvoyer ← (i≤n)
Fin

```

Paramètres Formels :

```

n : Entier (E)
elt : TElement (E)
tab : Tableau (E)

```

Variables intermédiaires :

```

i : entier

```

EXEMPLE4 (SUITE)

- **Remarque :**
- Le nombre de comparaisons dépend de la nature des données dans le tableau
- ➡ On distingue 3 cas :
- **Cas favorable :** Cas où l'élément recherché est aussi le premier élément du tableau . Dans ce cas $C_{Min}(n)=1$.
- **Cas défavorable :** Cas où l'élément recherché est le dernier élément du tableau. Dans ce cas $C_{Max}(n)=n$.

EXEMPLE4 (SUITE)

- **Moyenne des cas :**
- **Remarque :** Si l'élément apparaît à la i ème position, la fonction effectue i comparaisons. Par contre, il effectue n comparaisons si l'élément recherché n'existe pas dans le tableau.
- Calculer le nombre moyen de comparaisons : Le nombre moyen de comparaisons pour une recherche est alors : $C_Moy(n) = \sum p_i * C_i$
- où, C_i est le coût de l'élément qui apparaît à la i ème position.
- Soient, p_i la probabilité pour que l'élément recherché apparaît à la i ème position ($i=1, \dots, i=n$).
- et q la probabilité pour que l'élément recherché n'appartient pas au tableau.
- On sait que $p_1 + p_2 + \dots + p_n + q = 1$

EXEMPLE4 (SUITE)

- **Hypothèse :**
- Si on suppose que toutes les positions du tableau où peut se trouver l'élément recherché sont équiprobables.
- **a) Si on sait que l'élément appartient au tableau,**
- alors $q=0$ et $p_i = 1/n$.
- Dans ce cas, $C_Moy(n) = \sum i * (1/n)$
- $= (1/n) * \sum i$
- $= (1/n) * (n(n+1)/2) = (n+1)/2 = \text{Linéaire}$
- $= \text{ordre de } n$

EXEMPLE4 (SUITE)

- **Hypothèse :**
- Si on suppose que toutes les positions du tableau où peut se trouver l'élément recherché sont équiprobables.
- **b)** Si on suppose que l'élément recherché a une chance sur 2 d'appartenir au tableau.
- Dans ce cas on a $q=1/2$, $p_1+..+p_n=1/2$, $C_i=i$ et $C_q=n+1$
- Or par hypothèse, $p_1=..=p_n$.
- Par suite $p_i=1/(2n)$
- Donc, $C_Moy(n) = \sum i * (1/2n) + (n+1)/2 =$ fonction linéaire = ordre de n

COMPLEXITÉS DES ALGORITHMES RÉCURSIFS**RAPPEL : APPROCHE DIVISER POUR MIEUX RÉGNER**

- *L'approche diviser pour mieux régner est basée sur le principe de division : il divise un problème en plusieurs sous problèmes similaires au problème initial, mais de taille inférieure, résolvent les sous problèmes de façon récursive, puis combinent ses solutions pour retrouver une solution au problème initial.*
- *L'approche 'Diviser pour Mieux Régner' donne lieu à trois étapes à chaque niveau de récursivité :*
- **Diviser** le problème en un certain nombre de sous problèmes
- **Régner** sur les sous problèmes en les résolvant récursivement. Par ailleurs, si la taille d'un sous problème est assez réduite, on peut le résoudre directement
- **Combiner** les solutions des sous problèmes en une solution complète pour le problème initial.

COMPLEXITÉS DES ALGORITHMES RÉCURSIFS

La complexité d'un algorithme récursif qui décompose un problème T en p sous-problèmes T_i s'exprime par une relation de récurrence :

$$\text{coût}(T) = \begin{array}{l} \text{coût(décomposition } T) \\ + \sum_{i=1}^p \text{coût}(T_i) \\ + \text{coût(fusion de } p \text{ solutions)} \end{array}$$

$$\text{coût(problème élémentaire)} = \dots$$

- *Il faut donc savoir résoudre des récurrences pour calculer la complexité*

EXEMPLE CALCUL DE N!

```
Fonction fact(n) → Entier
P.F n : Entier (E )
début
    Si (n = 0) alors
        Renvoyer 1
    Sinon
        Renvoyer n * fact(n-1)
    Finsi
Fin
```

Donc

EQUATION DE LA COMPLEXITÉ DE N!

Remarque

- Le calcul de la factorielle ne dépend pas de la nature des nombres à multiplier, mais elle dépend uniquement de l'entier n (qu'est une entrée de l'algorithme).
 - Donc $C_Min(n) = C_Moy(n) = C_Max(n) = C(n)$
- Si on suppose que $C(n)$ est la complexité de la fonction factorielle en nombre de multiplications, alors $C(n-1)$ est celle de l'appel récursif.
- Or à chaque appel, on effectue une multiplication.
- Par suite $C(n) = C(n-1) + 1$
- Et $C(0) = 0$ Puisque, on effectue aucune multiplication si $n=0$.

Donc

$$C(n) = \begin{cases} C(n-1) + 1 & \text{si } n > 0 \\ 0 & \text{si non} \end{cases}$$

Donc

RÉSOLUTION DE L'EQUATION DE RÉCURRENCE

Résolution

$$C(n) = \begin{cases} C(n-1) + 1 & \text{si } n > 0 \\ 0 & \text{si non} \end{cases}$$

On effectue la résolution de l'équation de récurrence ci-dessus par sommation :

- $C(n) = C(n-1) + 1$
- $C(n-1) = C(n-2) + 1$
- ...
- $C(1) = C(0) + 1$
- -----
- $C(n) = 1 + \dots + 1 = n$ = **Fonction linéaire.**

PRINCIPALES CLASSES DE COMPLEXITÉ

- Lors de l'analyse de complexité, on essaie de se ramener aux classes suivantes :
 - **Complexité logarithmique** : coût en $\ln(n)$.
 - **Exemple** : recherche dichotomique dans un tableau trié $tab[1..n]$.
 - **Complexité linéaire** : coût linéaire.
 - **Exemple** : calcul du produit scalaire de deux vecteurs de $\mathbb{R}^n$
 - **Complexité quasi-linéaire** : coût en $(n \ln n)$.
 - **Exemple** : tri par fusion.
 - **Complexité polynomiale** : coût en (n^k) , avec $k > 1$.
 - **Exemple** : multiplication de deux matrices carrées d'ordre n : coût en (n^3) .
 - **Complexité exponentielle** : coût en (a^n) , avec $a > 1$.
 - **Exemple** : tours de Hanoï. Coût en 3^n